Reinforcement Learning

Lecture 1
Introduction & Exploration vs Exploitation

Stergios Christodoulidis

MICS Laboratory
CentraleSupélec
Université Paris-Saclay

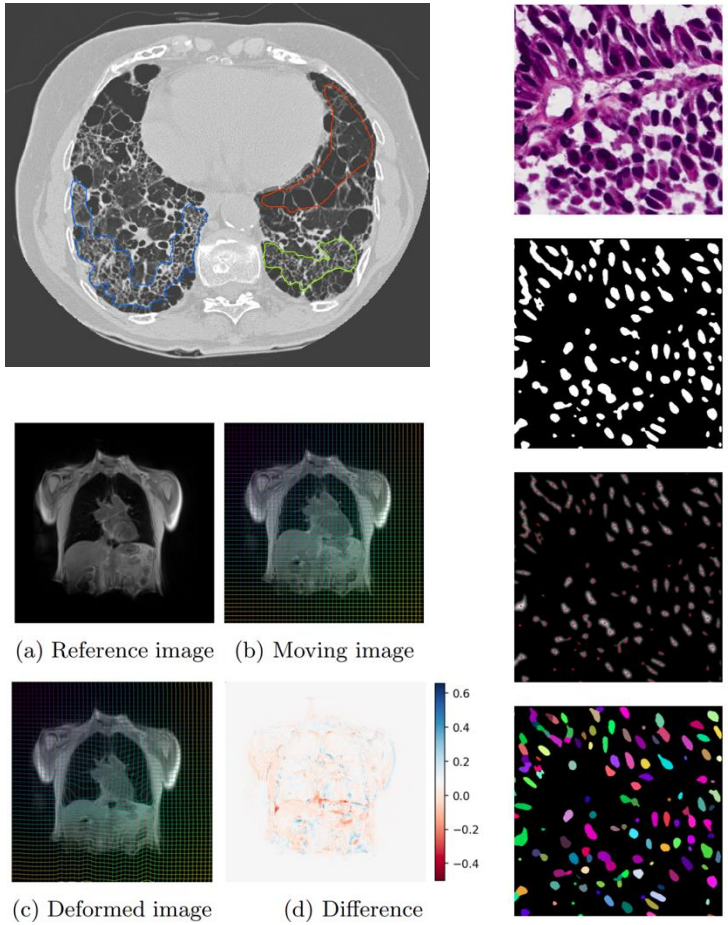
https://stergioc.github.io/





About Me

- PhD degree in University of Bern
  - Medical Image Analysis
  - Diagnosis Support Systems
- Postdoctoral Researcher at GR
  - AI for Cancer
  - Treatment Decision Support
  - Understanding of biological aspects
- Assistant Prof at CS MICS Lab
  - Artificial Intelligence
  - Healthcare Applications



The Team



Stergios Christodoulidis, PhD

βiomathematics Research Group, MICS Laboratory

Office: Bâtiment Bouygues sb.132

Email: stergios.christodoulidis@centralesupelec.fr



Pierre Marza, PhD

βiomathematics Research Group, MICS Laboratory

Office: Bâtiment Bouygues, MICS Laboratory

Email: pierre.marza@centralesupelec.fr

Irene Gentilini

βiomathematics Research Group, MICS Laboratory

Office: Bâtiment Bouygues, MICS Laboratory

Email: **i**rene.gentilini@centralesupelec.fr

Admin

- Course starts at the 30/01/2026 and ends at 31/03/2026
- Friday Lectures 13:45 – 17:00 (3h, 15 minutes break)
- Information, slides and announcements will be posted at Edunao:
  - https://centralesupelec.edunao.com/course/view.php?id=10316

➢ Office hours: we will be available right after the lecture or, send us an email and we will find a good time to meet

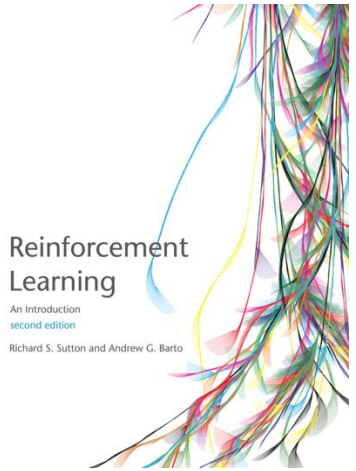
Learning Objectives of the Course

By the end of the course, you will be able to:

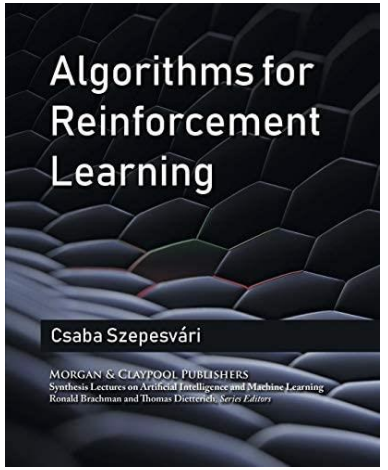
- Summarize the major aspects of the Reinforcement Learning (RL) problem.
- Describe the core RL concepts like the agents, environment, value functions, policies, etc.
- Translate real world problems under the RL formalism.
- Identify the most appropriate algorithm to solve a specific RL problem.
- Develop and compare classical algorithms in python using publicly available libraries.

Reading Material

- An Introduction to Reinforcement Learning,
  - Sutton and Barto, MIT Press, 1998
  - Available free online!
  - http://www.incompleteideas.net/book/the-book-2nd.html



- Algorithms for Reinforcement Learning,
  - Csaba Szepesvari, Morgan and Claypool, 2010
  - Available free online!
  - https://sites.ualberta.ca/~szepesva/rlbook.html



Organization of the Course

- The course will be split into Lectures and Lab Sessions
- The assessment will be based on:
  - Lab Assignments
  - An Individual Assignement
  - A Group Project of 2 people (max 3)

Schedule of the Lectures

- Lecture - Introduction to Reinforcement Learning & Exploration vs Exploitation (3h00)
- Lecture – Markov Decision Processes (1h30) & Lab – Exploration vs Exploitation (1h30)
- Lecture & Lab – Dynamic Programming (1h30 & 1h30)

-- [Winter break – two weeks] --

- Lecture & Lab – Model Free Prediction (1h30 & 1h30)
- Lecture & Lab – Model Free Control (1h30 & 1h30)
- Lecture & Lab – Value Function Approximation (1h30 & 1h30)
- Lecture & Lab – Policy Gradient methods (1h30 & 1h30)
- Lecture – Recent Advances (1h) & Final Presentation Session – Group Projects (2h00)

Due Dates

- Lab Reports Deliverables Submission: **One week after the lab session!**
- Group Project Proposals: **20/02/2026**

-- [Winter break – two weeks] --

- Group Project Poster Presentation and Individual Assignment: **31/03/2026 (End of the Course)**
- Group Project Final Submission: **03/04/2026 (One week after last lecture)**
- Late Submission Policy: **-10% (No deadline extensions will be given!)**

Assessment in Detail

- Assessment will be based on lab sessions deliverables (20%), individual assignment (20%), group project (60%) and for submited lecture notes (extra 10%).
  1. Labs:
     - All labs sessions assignments need to be submitted (20%)
  2. Individual Assignment
     - Final Submission (2-3 pages report, code, 20%)
  3. Group Project:
     - Project proposal document and presentation (single page document, 10%)
     - Poster Presentation (poster/presentation, 10%)
     - Final Submission (5-6 pages report, code, 40%)
  4. Lecture Notes:
     - Submited in any digital format (.md, .tex, .doc, +10%)

Acknowledments

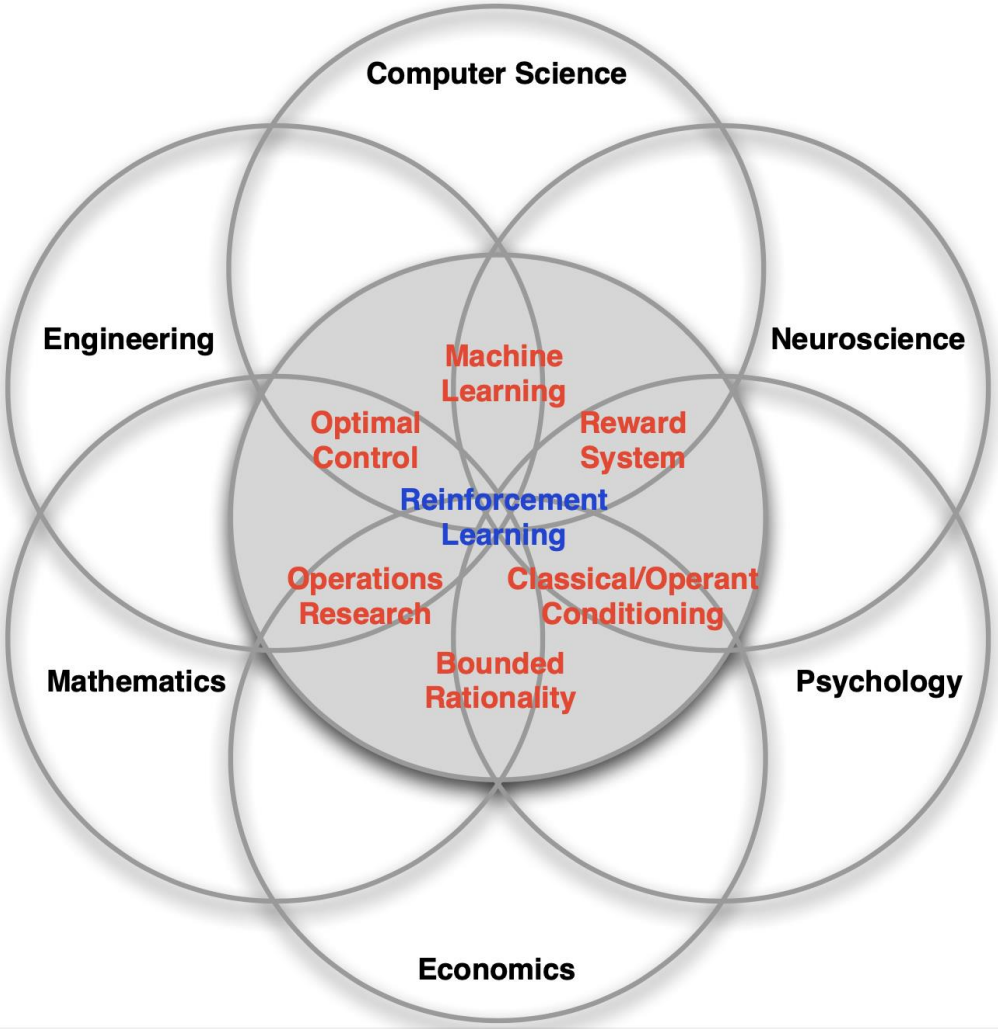
- The lecture slides are based on the structure and material by:
  - David Silver (Deep Mind, UCL)
  - Hado van Hasselt (Deep Mind)
  - An Introduction to Reinforcement Learning (Sutton and Barto)
  - https://www.deepmind.com/learning-resources

Today's Lecture

- Introduction to Reinforcement Learning
- The Reinforcement Learning problem
- Inside an RL agent
- Examples
- Exploration vs Exploitation
- Multi-Armed Bandits
- Greedy and ε-Greedy Agents
- Policy Gradients
- UCB Algorithms

What is Reinforcement Learning?

What is Reinforcement Learning?



[David Silver, IRL, UCL 2015]

What is Reinforcement Learning?

- Science and framework of learning to make decisions from interaction
- This requires us to think about
  - ...time
  - ...(long-term) consequences of actions
  - ...actively gathering experience
  - ...predicting the future
  - ...dealing with uncertainty
- Huge potential scope
- A formalization of the AI problem

What is Reinforcement Learning?

- People and animals learn by interacting with their environment.
- This type of learning can be defined as active learning and it based on sequencial interactions
  - Future interactions depend on earlier ones
- The learning is typically goal-oriented
- Instead of learning with examples, a reward is optimized.

RL characteristics

- There is no supervision, only a reward signal.
- Feedback might be delayed.
- Time matters (sequential, non i.i.d. data)
  - Agent's actions affect the subsequent data it receives

Why Reinforecement Learning?

There are distinct reasons to learn:

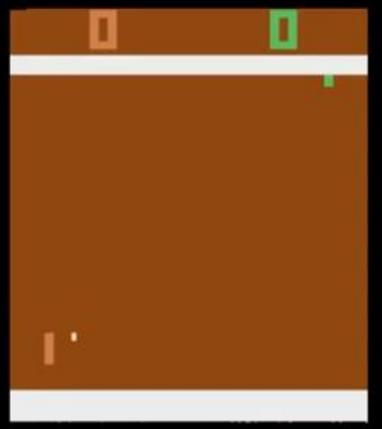
1. Find solutions
   - A program that plays chess well
   - A manufacturing robot with a specific purpose
2. Adapt online, deal with unforeseen circumstances
   - A chess program that can learn to adapt to you
   - A robot that can learn to navigate unknown terrains

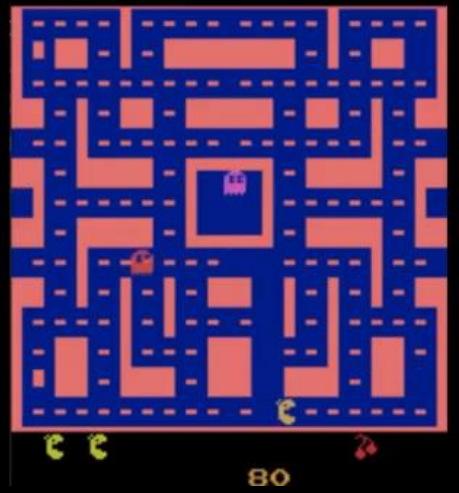
- Reinforcement learning can provide algorithms for both cases
- Note that the second point is not (just) about generalization — it is about continuing to learn efficiently online, during operation


205 2 1


0
OXYGEN
ACTIVISION




0 0


80


99
10600
3
PLAYER
ENEMY


0121
2 299
ACTIVISION


0045 0000

Reward

- A reward R_t is a scalar feedback signal
- Indicates how well agent is doing at step t — defines the goal
- The agent's goal is to maximize cumulative reward
- The Reinforcement Learning paradigm is based on the reward hypothesis

Hypothesis
Any goal can be formalized as the outcome of maximizing a cumulative reward

Examples of Rewards

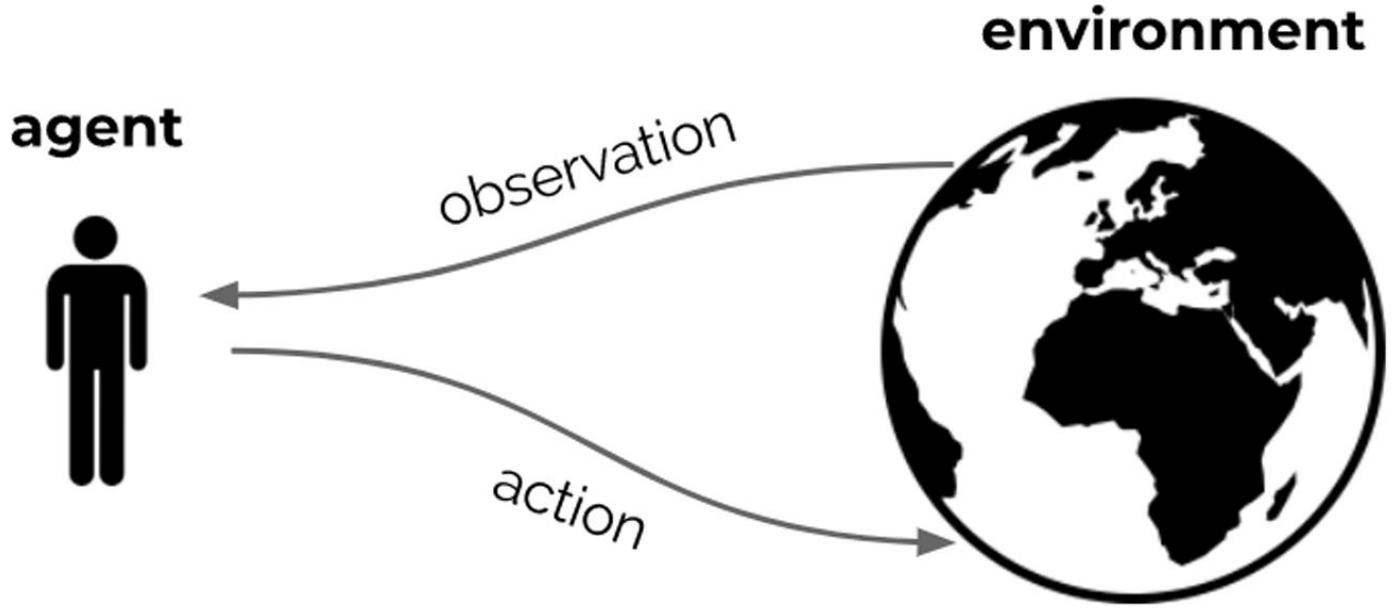
- Fly a helicopter
  - Reward: air time (+), crashing (-) …
- Manage an investment portfolio
  - Reward: gain (+), …
- Control a heating system
  - Reward: efficiency (+), cost (-), …
- Make a robot walk
  - Reward: distance covered (+), speed (+/-), out of battery (-) …
- Play video games
  - Reward: score (+), win (+), completion time (-)…

Sequential Decision Making

- Goal: *select actions to maximize total future reward*
- Actions may have long term consequences
- Reward may be delayed
- It may be better to sacrifice immediate reward to gain more long-term reward
- Examples:
  - A financial investment (may take months to mature)
  - Refueling a helicopter (might prevent a crash in several hours)
  - Blocking opponent moves (might help winning chances many moves from now)

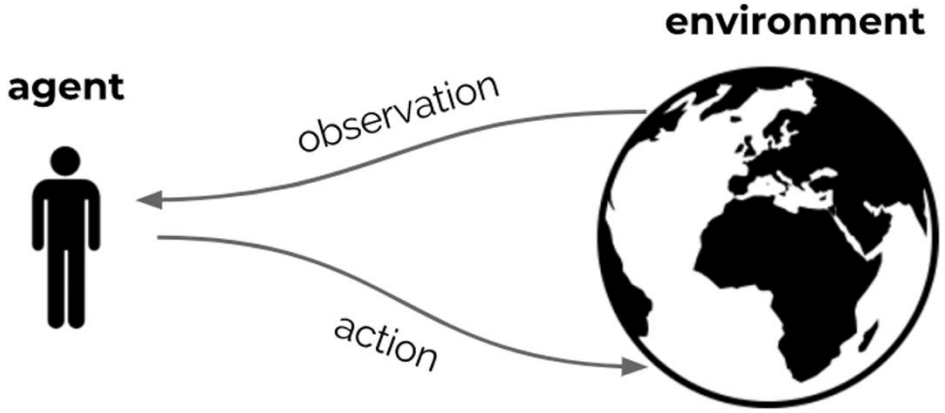
Formalizing the Reinforcement Learning Problem

The RL Problem



[Hado van Hasselt, 2021]

The Agent and the Environment



- At each step t the agent:
  - Receives observation O_t (and reward R_t)
  - Executes action A_t
- The environment:
  - Receives action A_t
  - Emits observation O_{t+1} (and reward R_{t+1})

[Hado van Hasselt, 2021]

History and State

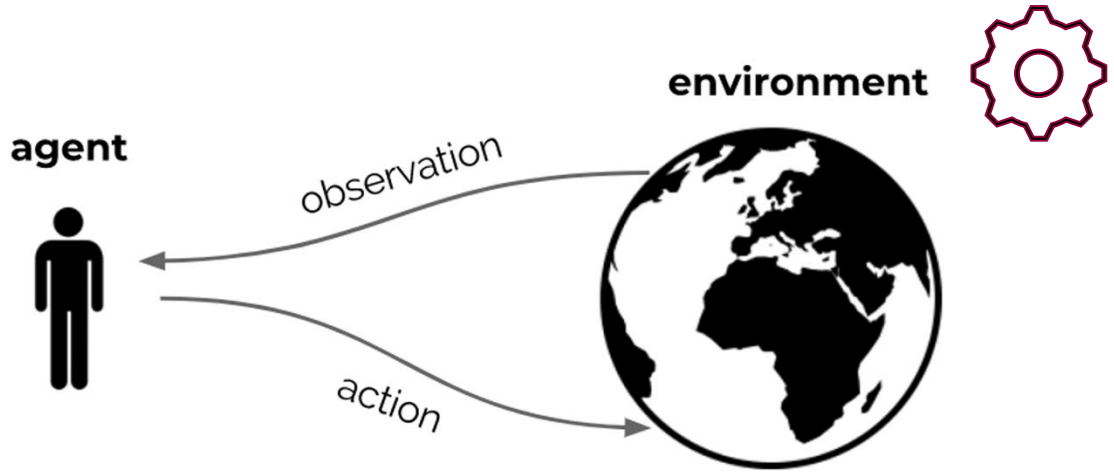
- The history is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$$

- i.e., all observable variables up to time t
- For instance, the sensorimotor stream of a robot
- This history is used to construct the agent state S_t
- State is the information used to determine what happens next
- Formally, state is a function of the history:

$$S_t = f(H_t)$$

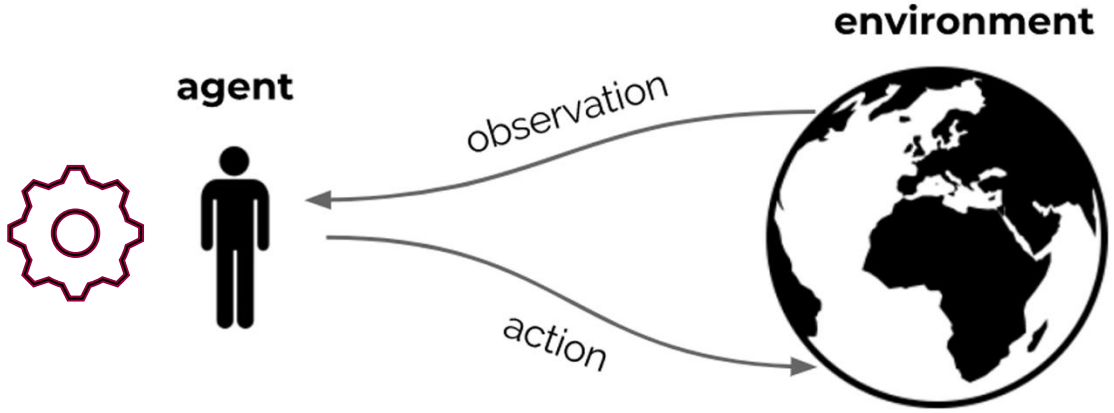
Environment State



- The environment state S_t^e is the environment's internal representation
  - i.e., whatever data the environment uses to pick the next observation/reward
- It is usually invisible to the agent
- Even if S_t^e is visible, it may contain lots of irrelevant information

[Hado van Hasselt, 2021]

Agent State



- The agent state S_t^a is the agent's internal representation
  - i.e., whatever data the agent uses to pick the next action
- It is the information that is used by reinforcement learning algorithms
- It can be any function of history:

$$S_t^a = f(H_t)$$

[Hado van Hasselt, 2021]

Information or Markov State

- An information state (a.k.a. Markov state) contains all useful information from the history.

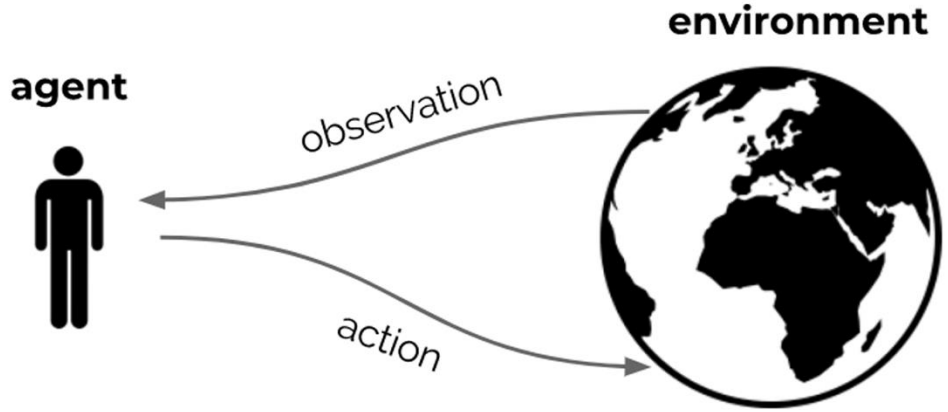
Definition

A state S_t is Markov if and only if:

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1 \dots S_t]$$

- "The future is independent of the past given the present"
- Doesn't mean it contains everything, just that adding more history doesn't help
- Once the state is known, the history may be thrown away
- The environment state S_t is Markov
- The history H_t is Markov

Fully Observable Environments



Full observability

- Suppose the agent sees the full environment state
  - Agent state = environment state = information state
  - Formally, this is a Markov decision process (MDP)

$$S_t^a = S_t^e = O_t$$

[Hado van Hasselt, 2021]

Partially Observable Environments

- Partial observability means that agent indirectly observes environment:
  - A robot with camera vision isn't told its absolute location
  - A trading agent only observes current prices
  - A poker playing agent only observes public cards
- Now agent state ≠ environment state
- Formally this is a partially observable Markov decision process (POMDP)
- Agent must construct its own state representation S_t^a, e.g.
  - Complete history: $S_t^a = H_t$
  - Beliefs of environment state: $S_t^a = (\mathbb{P}[S_t^e = s^1], \dots, \mathbb{P}[S_t^e = s^n])$
  - Recurrent neural network: $S_t^a = \sigma(S_{t-1}^a W_s + O_t W_o)$

Major Components of an RL Agent

Major Components of an RL Agent

- An RL agent may include one or more of these components:
  - Policy: agent's behavior function
  - Value function: how good is each state and/or action
  - Model: agent's representation of the environment

Policy

- A policy is the agent's behavior
- It is a map from state to action, e.g.
  - Deterministic policy: $a = \pi(s)$
  - Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$

Value Function

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- We call the expected cumulative reward (return), from a state s, the value

$$\begin{aligned} \mathrm{v}_\pi(s) &= \mathbb{E}\left[G_t | S_t = s\right] \\ &= \mathbb{E}\left[R_{t+1} + R_{t+2} + R_{t+3} + \cdots \,| S_t = s\right] \end{aligned}$$

- The value depends on the actions the agent takes
- Goal is to maximize value, by picking suitable actions
- Rewards and values define utility of states and action (no supervised feedback)
- Returns and values can be defined recursively

$$\begin{aligned} G_t &= R_{t+1} + G_{t+1} \\ \mathrm{v}_\pi(s) &= \mathbb{E}\left[R_{t+1} + \mathrm{v}(S_{t+1}) | S_t = s\right] \end{aligned}$$

Action Values

- It is also possible to condition the value on actions:

$$q(s,a) = \mathbb{E}\left[G_t | S_t = s, A_t = a\right]$$

Model

- A model predicts what the environment will do next
- $\mathcal{P}$ predicts the next state
- $\mathcal{R}$ predicts the next (immediate) reward, e.g.

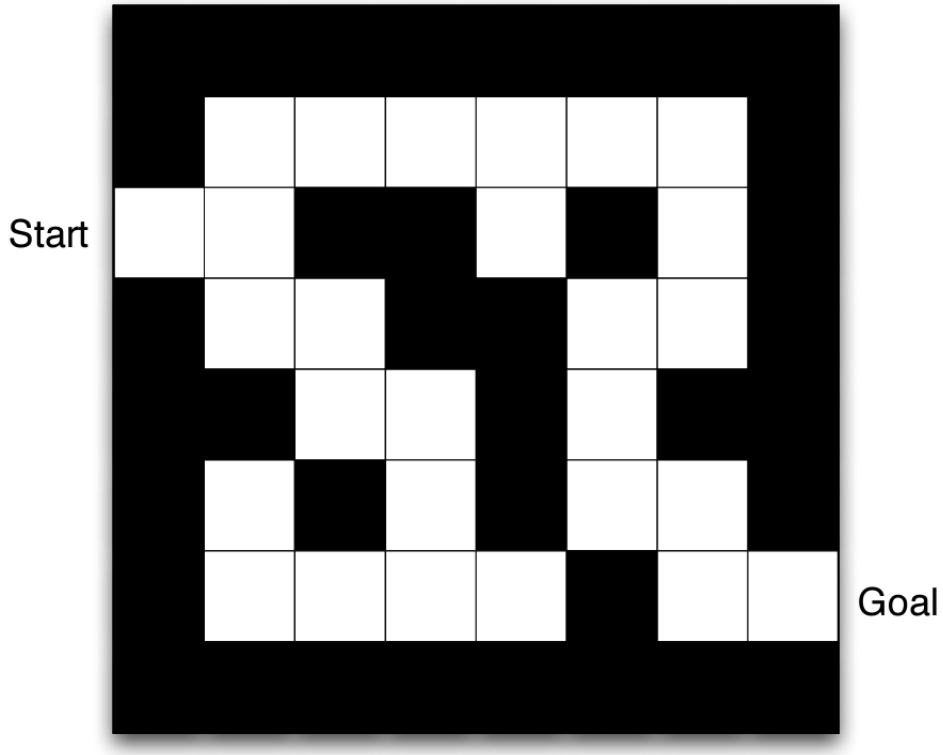
$$\mathcal{P}_{ss'}^{a} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$

$$\mathcal{R}_{s}^{a} = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

Maze Example

Maze Example (1/4)

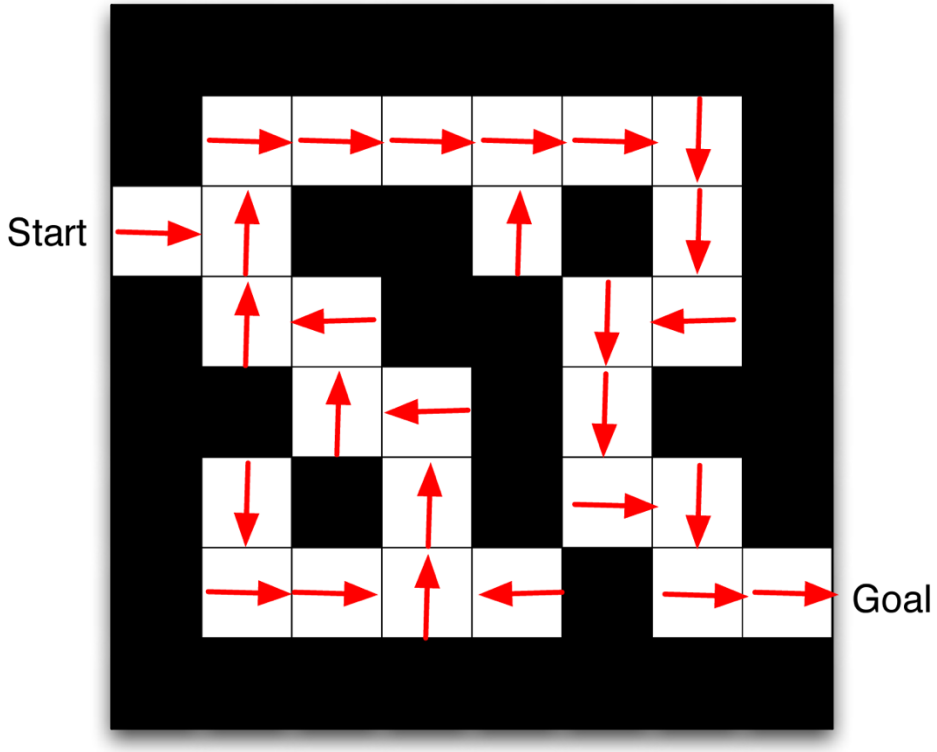
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location



[Hado van Hasselt, 2021]

Maze Example (2/4)

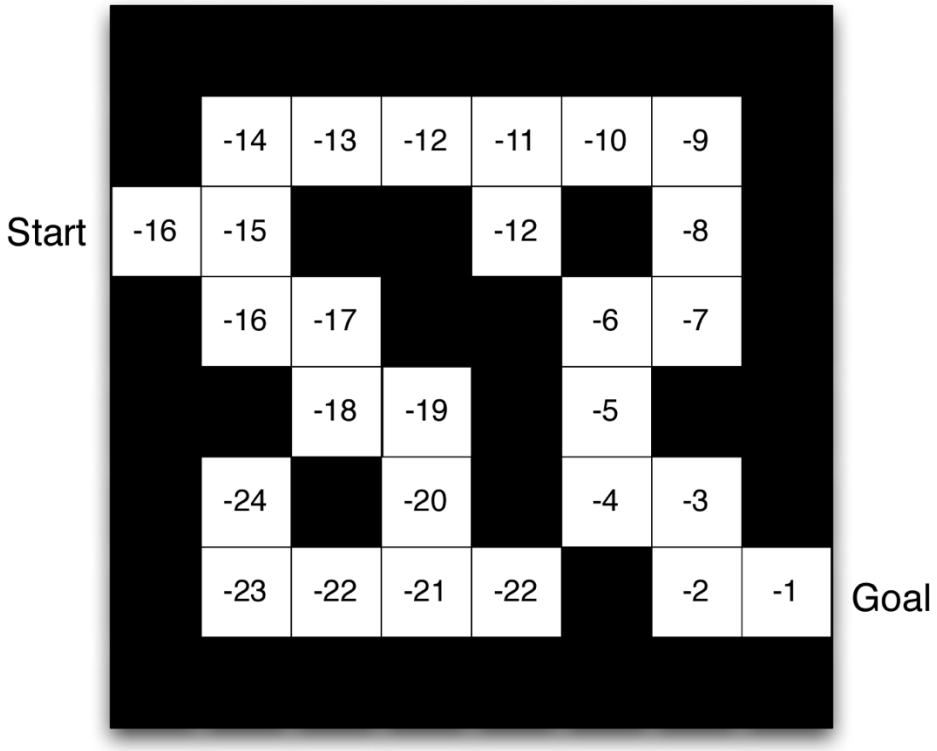
- Arrows represent policy $\pi(s)$ for each state s



[Hado van Hasselt, 2021]

Maze Example (3/4)

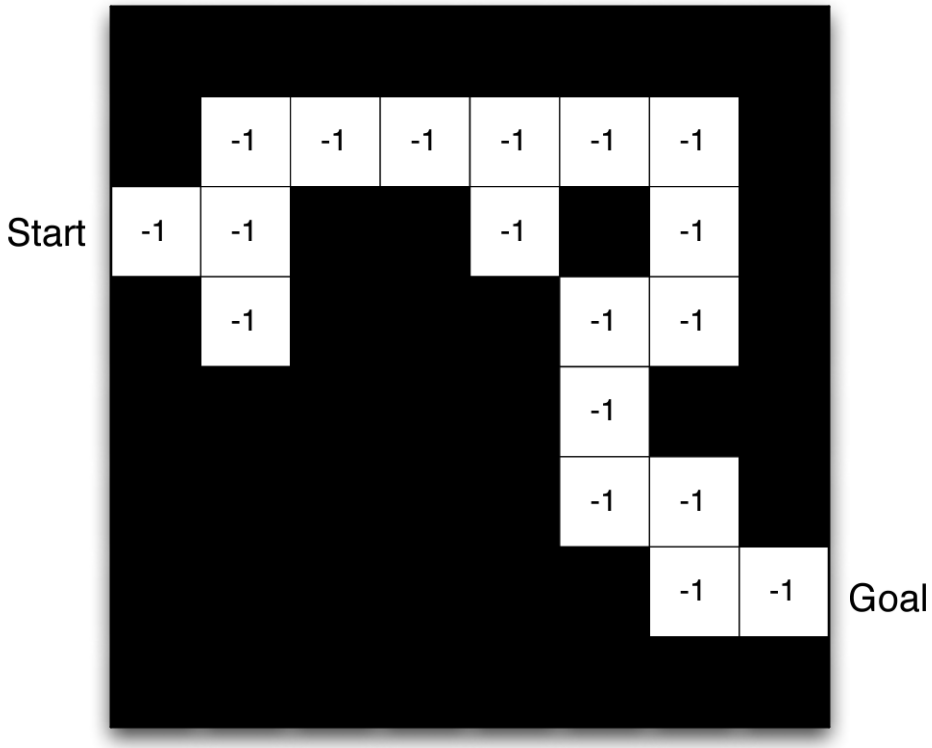
- Numbers represent value $v_\pi(s)$ of each state s



[Hado van Hasselt, 2021]

Maze Example (4/4)

- Agent may have an internal model of the environment
- Dynamics: how actions change the state
- Rewards: how much reward from each state
- The model may be imperfect
- Grid layout represents transition model $\mathcal{P}_{ss'}^{a}$
- Numbers represent immediate reward R_s^a from each state s (same for all a)



[Hado van Hasselt, 2021]

Agent Categories (1/2)

- Value Based
  - No Policy (Implicit)
  - Value Function
- Policy Based
  - Policy
  - No Value Function
- Actor Critic
  - Policy
  - Value Function

Agent Categories (2/2)

- Model Free
  - Policy and/or Value Function
  - No Model
- Model Based
  - Optionally Policy and/or Value Function
  - Model

Subproblems of the RL problem

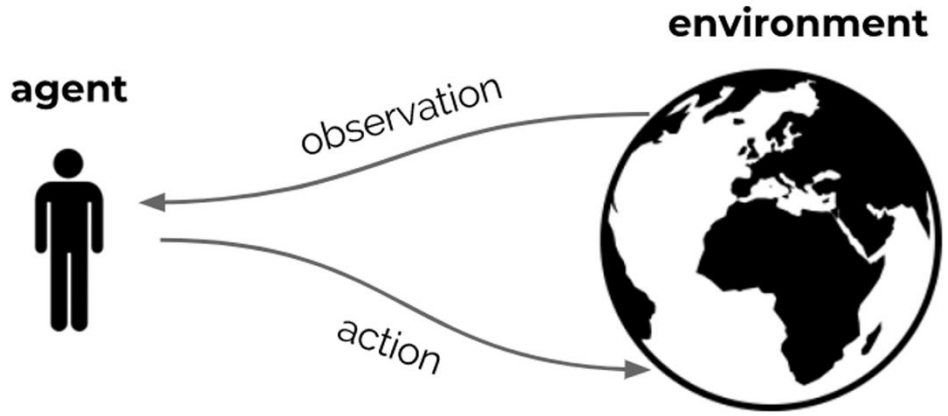
Prediction and Control

- Prediction: evaluate the future (for a given policy)
- Control: optimize the future (find the best policy)
- These can be strongly related.
- If we could predict everything, do we need anything else?

Learning Agent Components

- All components are functions
  - Policies, π: $\mathcal{S} \rightarrow \mathcal{A}$ (or to probabilities over $\mathcal{A}$)
  - Value functions, v: $\mathcal{S} \rightarrow R$
  - Models, m: $\mathcal{S} \rightarrow \mathcal{S}$ and/or r: $\mathcal{S} \rightarrow R$
- E.g., we can use neural networks, and machine learning techniques to learn them.
- Deep reinforcement learning is a rich and active research field

Introduction Summary



- Reinforcement learning is the science of learning to make decisions
- Agents can learn a policy, value function and/or a model
- The general problem involves taking into account time and consequences
- Decisions affect the reward, the agent state, and environment state
- Learning is active: decisions impact data

[Hado van Hasselt, 2021]

Tabular Solution Methods

- At the beginning will consider quite small problems
- In such problems the action and state spaces are quite small
- Therefore, the approximations of the state/action value functions can be represented as arrays or matrices (lookup tables)
- E.g.,

$$Q(s,a) = \begin{bmatrix} q(s_1,a_1) & \cdots & q(s_1,a_n) \\ \vdots & \ddots & \vdots \\ q(s_m,a_1) & \cdots & q(s_m,a_n) \end{bmatrix}$$

- So, the task would be to approximate this matrix.
- In such cases, the methods can often find exact (optimal) solutions.

Approximate Solution Methods

- Later, we will increase the dimensionality.
- In such problems the action and state spaces are very large to be represented with lookup tables.
- Therefore, for the approximations of the state/action value more general functions will be used.
- In such cases, the methods can only find approximate solutions.

Exploration vs. Exploitation

Exploration vs. Exploitation Dilemma

- Online decision-making involves a fundamental choice:
  - Exploitation Make the best decision given current information
  - Exploration Gather more information
- The best long-term strategy may involve short-term sacrifices
- Gather enough information to make the best overall decisions

Examples

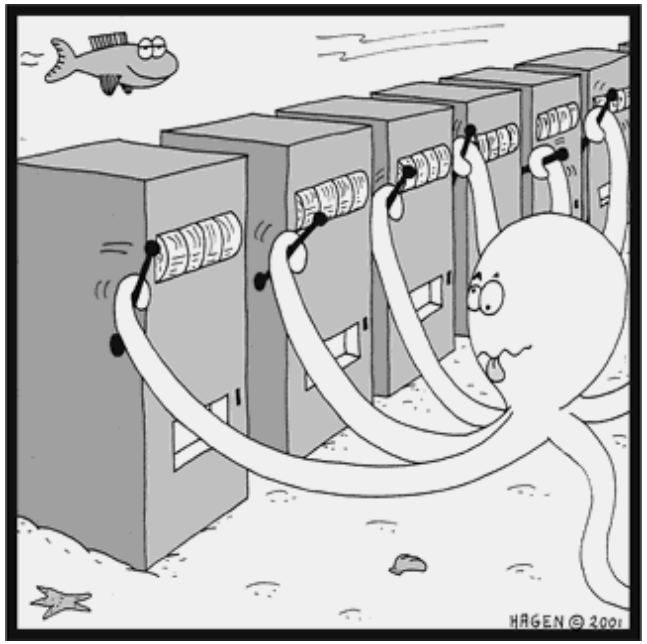
- Restaurant Selection
  - Exploitation Go to your favorite restaurant
  - Exploration Try a new restaurant
- Online Banner Advertisements
  - Exploitation Show the most successful advertisement
  - Exploration Show a different advertisement
- Oil Drilling
  - Exploitation Drill at the best-known location
  - Exploration Drill at a new location
- Game Playing
  - Exploitation Play the move you believe is best
  - Exploration Play an experimental move

Principles

- Naive Exploration
  - Add noise to greedy policy (e.g., ε-greedy)
- Optimistic Initialization
  - Assume the best until proven otherwise
- Optimism in the Face of Uncertainty
  - Prefer actions with uncertain values

Multi-Armed Bandits

- Consider the following learning problem:
  - You are faced repeatedly with a choice among k different options, or actions.
  - After each choice you receive a numerical reward chosen by a predefined distribution.
- Formally:
  - A multi-armed bandit is a set of distributions $< \mathcal{A}, \mathcal{R} >$
  - Single state case
  - $\mathcal{A}$ is a (known) set of actions (or “arms")
  - $\mathcal{R}^a(r) = \mathbb{P}[r|a]$ is an unknown probability distribution over rewards
  - At each step t the agent selects an action $a_t \in \mathcal{A}$
  - The environment generates a reward $r_t \sim \mathcal{R}^{a_t}$
  - The goal is to maximize cumulative reward $\sum_{\tau=1}^{t} r_\tau$



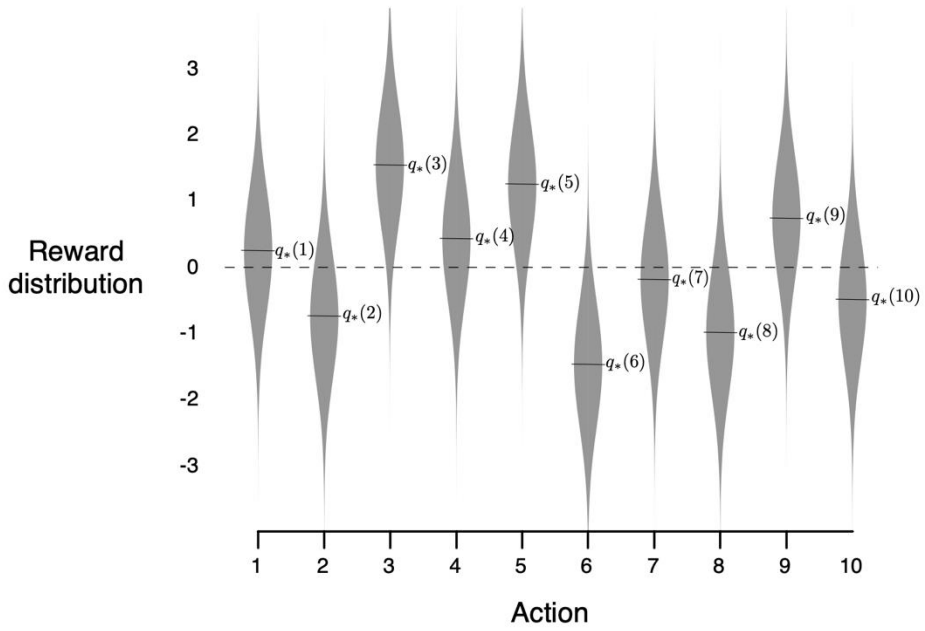
[David Silver, IRL, UCL 2015]

10-armed Testbed

- In our k-armed banded problem, each of the action has an expected or mean reward given that that action is selected
- We can say that the optimal value q_* is:

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

- This encodes the dynamics of our environment.
- We assume that we do not know the action values, but we can keep estimates.



[An Introduction to Reinforcement Learning, Sutton and Barto]

Values and Regret

- The action-value is the expected reward for action a,

$$Q(a) = \mathbb{E}[\mathcal{R} \mid A = a]$$

- The optimal value V^* is

$$\mathrm{V}^* = Q(a^*) = \max_{a \in \mathcal{A}} Q(a)$$

- The regret is the opportunity loss for one step

$$I_t = \mathbb{E}[\mathrm{V}^* - Q(a_t)]$$

- The regret for the optimal action is zero

Regret

- We want to minimize total regret:

$$L_t = \mathbb{E}\left[\sum_{\tau=1}^{t} \mathrm{V}^* - Q(a_\tau)\right]$$

- Maximize cumulative reward ≡ minimize total regret
- The summation spans over the full ‘lifetime of learning’

10-Armed Bandid Example

- What is value will look like in the case of a 10-armed bandit problem?

$$Q(a) = [q_1, q_2, \dots, q_{10}]$$

  - Single state problem, so only Q values can be defined
- What does it mean to get the max?

$$a = argmax(Q)$$

Action-value Methods

- How do we approximate Q in practice during training?
- One natural way to estimate this is by averaging the rewards received after we have selected a lot of them (law of large numbers):

$$Q_t(a) \doteq \frac{sum\ of\ rewards\ when\ a\ taken\ prior\ to\ t}{number\ of\ times\ a\ taken\ prior\ to\ t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

Incremental Implementation

- Is there a more efficient way to approximate these expectations?

$$Q_{n+1} = \frac{1}{n}\sum_{i=1}^{n} R_i = \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right)$$

$$= \frac{1}{n}\left(R_n + (n-1)\frac{1}{n-1}\sum_{i=1}^{n-1} R_i\right) = \frac{1}{n}(R_n + (n-1)Q_n)$$

$$= \frac{1}{n}(R_n + nQ_n - Q_n) = Q_n + \frac{1}{n}[R_n - Q_n]$$

$$New\ Estimate \leftarrow Old\ Estimate + Scaler[Target\ - Old\ Estimate]$$

Algorithms

- Considering that we have an estimate of the values we can use one of the following algorithms:
  - Greedy, ε-Greedy
  - UCB
  - Policy Gradient
- The main concepts of such algorithms can be used in order to solve more complex tasks.

Greedy Algorithms

Greedy Algorithm

- Considering that we have an estimate of the values one of the simplest algorithm is the greedy selection
- Select action with highest value:

$$A_t = \underset{a \in \mathcal{A}}{argmax} Q_t(a)$$

- Greedy can lock onto a suboptimal action forever

ε-Greedy Algorithm

- The ε-greedy algorithm:
  - With probability 1 −ε select greedy action: $A_t = \underset{a \in \mathcal{A}}{argmax}\, Q_t(a)$
  - With probability ε select a random action
- ε-greedy continues to explore

A Simple Bandit Algorithm

A simple bandit algorithm

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

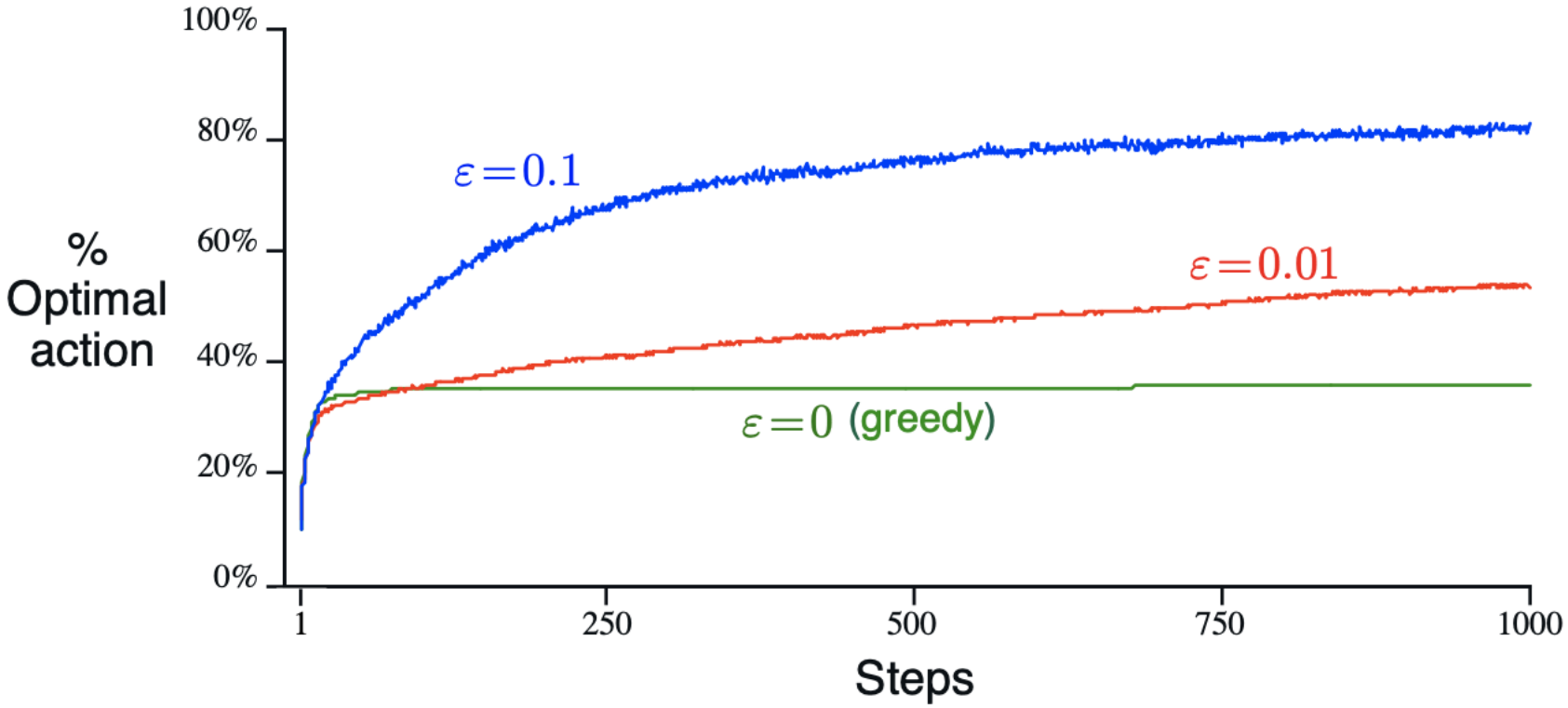
$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

[An Introduction to Reinforcement Learning, Sutton and Barto]

Greedy vs ε-Greedy

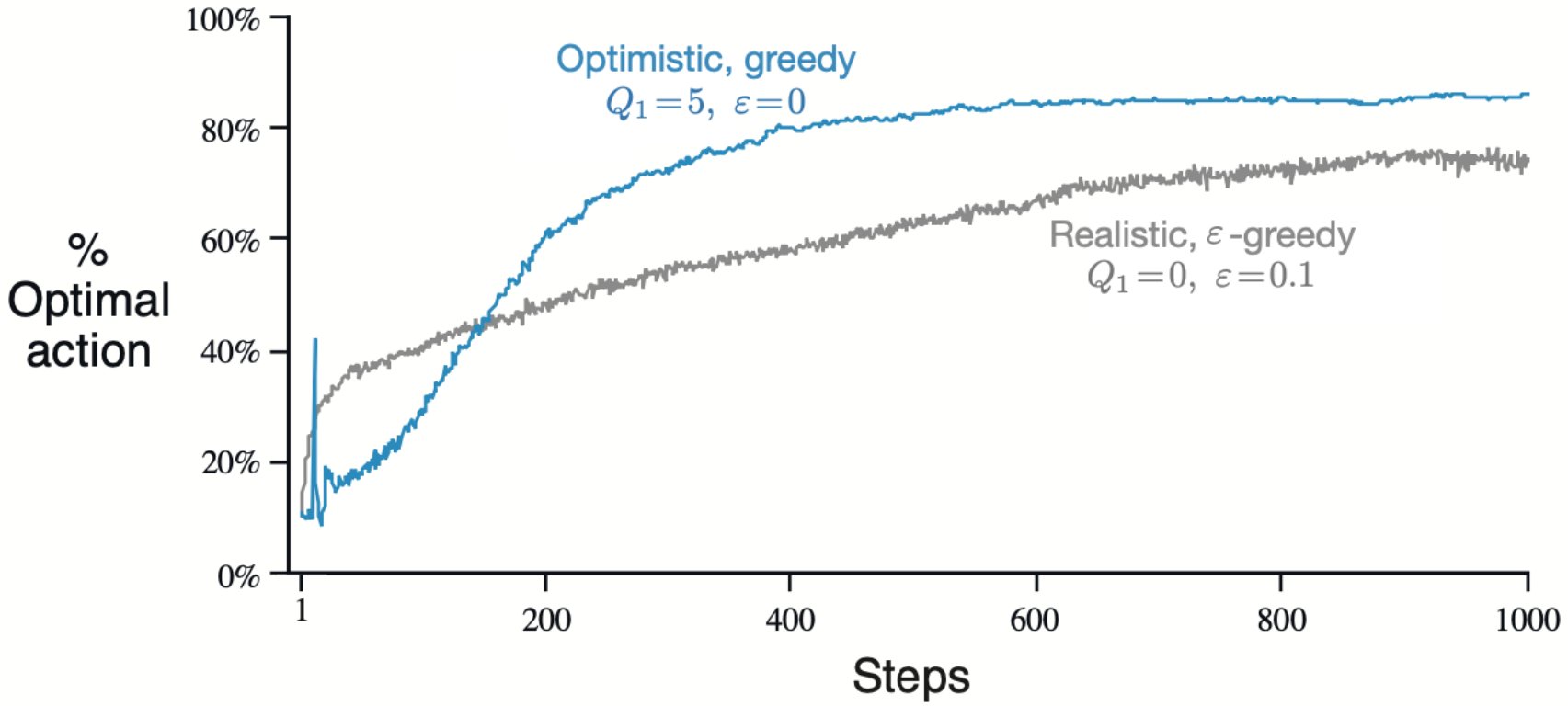


[An Introduction to Reinforcement Learning, Sutton and Barto]

Optimistic Initialization

- Simple and practical idea: initialize $Q(a)$ to high value
- Update action value at every time step
- Value decreases over time
- Encourages systematic exploration early on

Optimistic vs Realistic Initialization



[An Introduction to Reinforcement Learning, Sutton and Barto]

Upper Confidence Bound

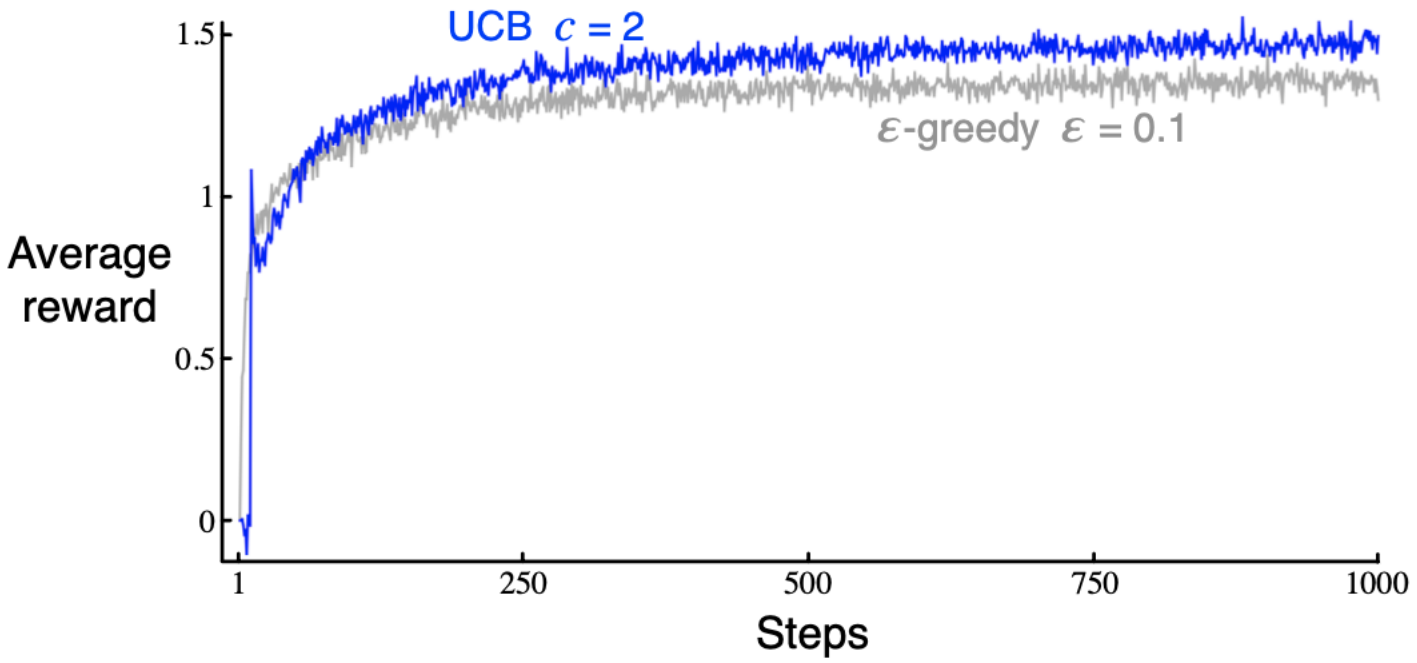
Upper Confidence Bound (UCB)

- The ε-Greedy algorithm performs exploration without any preference (random).
- Why not explore in a more explicit way?
- UCB selects the actions with the most uncertain value function estimates!

$$A_t \doteq \underset{a}{\text{argmax}} \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- $N_t(a)$ denotes the times action a was selected prior to time t
- Eventually, the square-root term is a measure of uncertainty

UCB vs ε-Greedy



[An Introduction to Reinforcement Learning, Sutton and Barto]

Policy Gradient

Policy Gradient Methods

- So far, we have considered methods that estimate action values to select actions.
- Then we select the actions with the highest action-value.
- Instead, can learn policies $\pi(a)$ directly?
- We can consider learning a numerical preference for each action:

$$H_t(a) \in \mathbb{R}$$

Gradient Bandits

- Considering a set of action preferences $H_t(a)$ we can define a policy using the softmax distribution.

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}}$$

- In this configuration, the larger the preference the more often this action is selected.

Stohastic Gradient Ascent

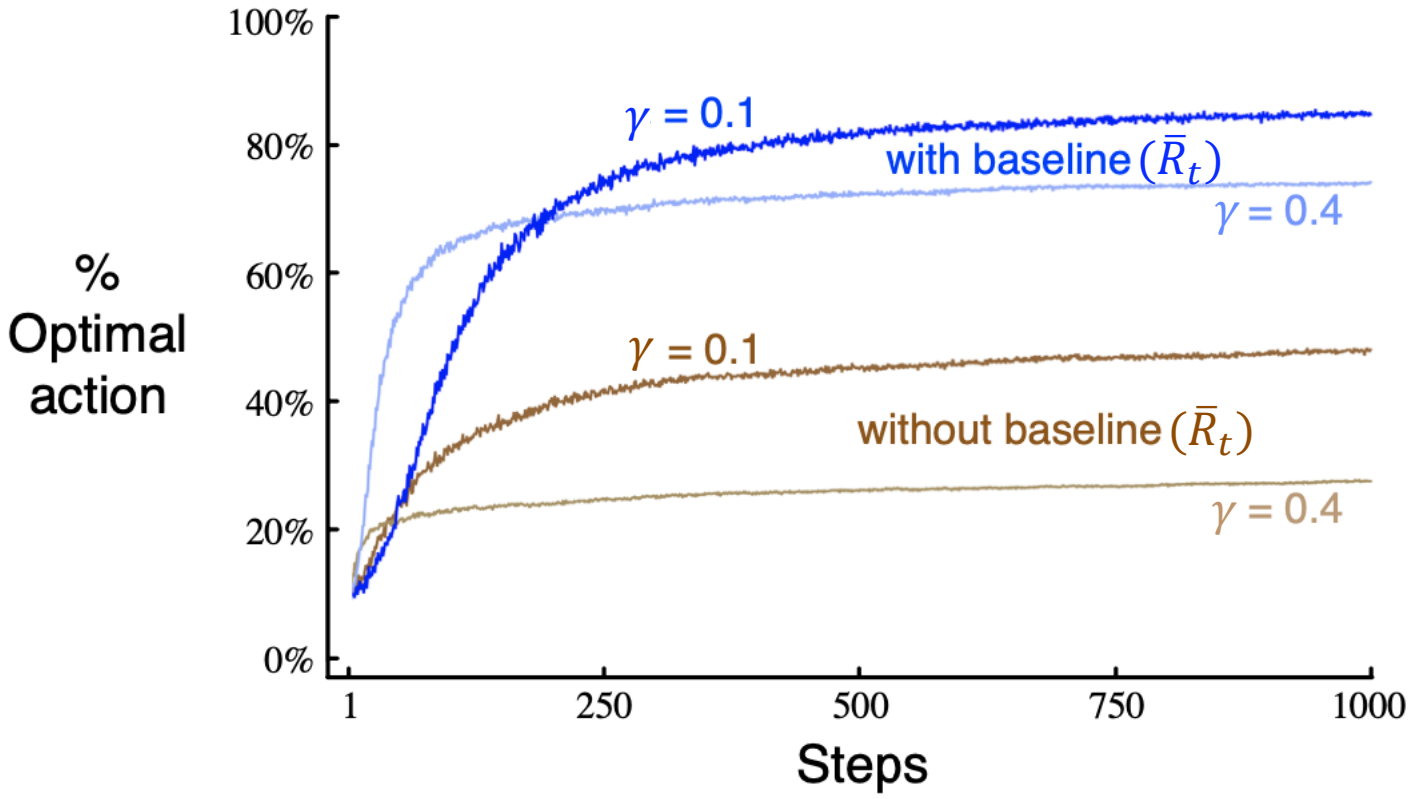
- There is a natural learning algorithm for soft-max actions preferences.
- This is the stochastic gradient ascent.
- On each step after selecting an action A_t and receiving a reward R_t the action preferences can be updated using:

$$H_{t+1}(A_t) = H_t(A_t) + \gamma(R_t - \bar{R}_t)\big(1 - \pi_t(A_t)\big)$$
$$H_{t+1}(a) = H_t(a) - \gamma(R_t - \bar{R}_t)\pi_t(a), \qquad for\ all\ a \neq A_t$$

- with $\bar{R}_t$ being the average of the rewards up to but not including time t (baseline)

Intuitively
If the reward is higher than the baseline then the probability of taking A_t is the future is increased, if the reward is lower than the baseline then the probability of taking A_t is decreased. All the reset actions move in the opposite direction

Gradient Bandits with and without Baseline



[An Introduction to Reinforcement Learning, Sutton and Barto]

Conclusion

- Have covered several principles for exploration/exploitation
- Each principle was developed in bandit setting
- Same principles can be extended to the MDP setting